Model Question Paper

QP CODE:
Reg No: _____
Name: _____

PAGES : 4

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY
EIGHTH SEMESTER B.TECH DEGREE EXAMINATION, MONTH & YEAR

Course Code: CST402

Course Name: Distributed Computing
Max.Marks:100
Duration: 3 Hours

PART A
Answer All Questions. Each Question Carries 3 Marks

1. Identify any three distributed applications and for each application, determine which all motivating factors are important for building an application over a distributed system.
2. Assume that the surface of the past cone form a consistent cut. Does it mean that all events on the surface of the past cone are always concurrent? Demonstrate with the help of an example.
3. Specify the issues in recording a global state.
4. Explain the rules used to update clocks in scalar time representation.
5. Describe how quorum-based mutual exclusion algorithms differ from the other categories of mutual exclusion algorithms.
6. Explain with example, how wait-for-graphs can be used in deadlock detection.
7. List any three advantages of using distributed shared memory.
8. Define the no -orphans consistency condition.
9. Define Byzantine agreement problem.
10. Differentiate between whole file serving and whole file caching in Andrew file system **(10x3=30)**

Part B
(Answer any one question from each module. Each question carries 14 Marks)

11. (a) Explain the three different models of the service provided by communication networks. **(6)**
    (b) Explain how the causal dependency between events in distributed execution is defined using Lamport's happened before relationship. **(8)**

OR

12. (a) Address the various strategies that can be adopted to satisfy the requirements of a reliable and fault tolerant distributed system. **(6)**

(b) Which are the different versions of send and receive primitives for distributed communication? Explain. **(8)**

13. (a) Illustrate bully algorithm for electing a new leader. Does the algorithm meet liveness and safety conditions? **(7)**

(b) Clearly mentioning assumptions, explain the rules of termination detection using distributed snapshots. **(7)**

OR

14. (a) In Chandy-Lamport algorithm for recording global snapshots, explain how the recorded local snapshots can be put together to create the global snapshot. Can multiple processes initiate the algorithm concurrently? **(6)**

(b) Illustrate the working of spanning tree based termination detection algorithm. **(8)**

15. (a) Explain and illustrate Lamport's mutual exclusion algorithm. **(8)**

(b) Discuss the three types of messages required for deadlock handling in Maekawa's algorithm. Explain how Maekawa's algorithm handles deadlocks. **(6)**

OR

(a) Explain and illustrate Ricart- Agrawala algorithm for achieving mutual exclusion. **(8)**

(b) Explain any three different models of deadlock. **(6)**

17. (a) What are the issues in failure recovery? Illustrate with suitable examples. **(7)**

(b) Show that Lamport's Bakery algorithm for shared memory mutual exclusion, satisfy the three requirements of critical section problem. **(7)**

OR

18. (a) Differentiate consistent and inconsistent states with examples. **(4)**

(b) What is check point-based rollback-recovery? Explain the three classifications of check point-based rollback-recovery. **(10)**

19. (a) Explain consensus algorithm for crash failures under synchronous systems. **(6)**

(b) Summarize distributed file system requirements. **(8)**

OR

20. (a) Differentiate Andrew file system and NFS. **(4)**

(b) Explain Sun NFS architecture with diagram. **(10)**

ANSWER KEY

1)

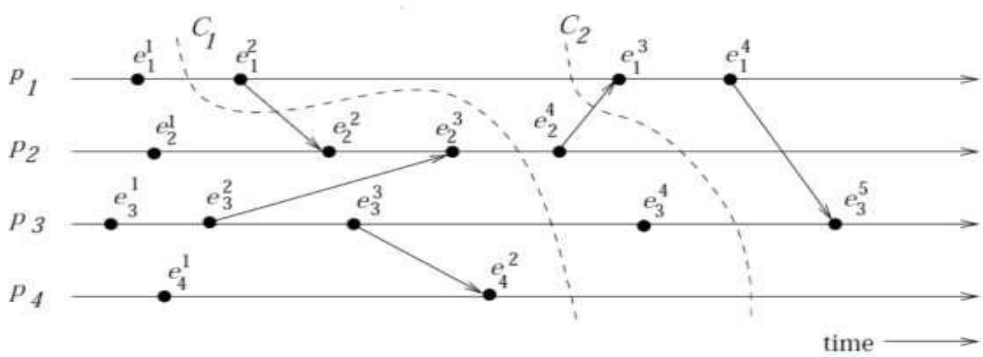
- Mobile systems
- Sensor networks
- Ubiquitous or pervasive computing

2)

"In the space-time diagram of a distributed computation, a *cut* is a zigzag line joining one arbitrary point on each process line."

- A cut slices the space-time diagram, and thus the set of events in the distributed computation, into a PAST and a FUTURE.
- The PAST contains all the events to the left of the cut and the FUTURE contains all the events to the right of the cut.
- For a cut C, let PAST(C) and FUTURE(C) denote the set of events in the PAST and FUTURE of C, respectively.
- Every cut corresponds to a global state and every global state can be graphically represented as a cut in the computation's space-time diagram.
- Cuts in a space-time diagram provide a powerful graphical aid in representing and reasoning about global states of a computation.

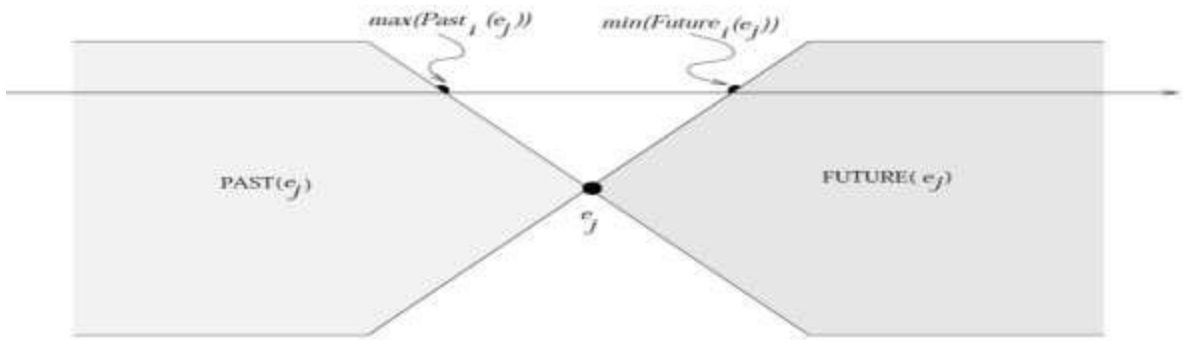
Figure 2.3: Illustration of cuts in a distributed execution.



Past Cone of an Event

- An event e_j could have been affected only by all events e_i such that $e_i \rightarrow e_j$.
- In this situtaion, all the information available at e_i could be made accessible at e_j.
- All such events e_i belong to the past of e_j.

Figure 2.4: Illustration of past and future cones.



Future Cone of an Event

- The future of an event e_j, denoted by $Future(e_j)$, contains all events e_i that are causally affected by e_j (see Figure 2.4).
- In a computation $(H, \rightarrow)$, $Future(e_j)$ is defined as:

$$Future(e_j) = \{e_i | \forall e_i \in H,\ e_j \rightarrow e_i\}.$$

3)

"A collection of the local states of its components, namely, the processes and the communication channels."

- The state of a process is defined by the contents of processor registers, stacks, local memory, etc. and depends on the local context of the distributed application.
- The state of channel is given by the set of messages in transit in the channel.
- The occurrence of events changes the states of respective processes and channels.
- An internal event changes the state of the process at which it occurs.
- A send event changes the state of the process that sends the message and the state of the channel on which the message is sent.
- A receive event changes the state of the process that or receives the message and the state of the channel on which the message is received.

A Channel State

- The state of a channel depends upon the states of the processes it connects.
- Let $SC_{ij}^{x,y}$ denote the state of a channel C_{ij}.

The state of a channel is defined as follows:

$$SC_{ij}^{x,y} = \{m_{ij} | \ send(m_{ij}) \leq e_i^x \bigwedge rec(m_{ij}) \not\leq e_j^y\}$$

Thus, channel state $SC_{ij}^{x,y}$ denotes all messages that p_i sent upto event e_i^x and which process p_j had not received until event e_j^y.

Global State

- The global state of a distributed system is a collection of the local states of the processes and the channels.
- Notationally, global state GS is defined as,

$$GS = \{\bigcup_i LS_i^{x_i}, \bigcup_{j,k} SC_{jk}^{y_j,z_k}\}$$

4)

Rules **R1** and **R2** to update the clocks are as follows:

- **R1** Before executing an event (send, receive, or internal), process p_i executes the following:

$$C_i := C_i + d \qquad (d > 0).$$

In general, every time **R1** is executed, d can have a different value, and this value may be application-dependent. However, typically d is kept at 1 because this is able to identify the time of each event uniquely at a process, while keeping the rate of increase of d to its lowest level.

- **R2** Each message piggybacks the clock value of its sender at sending time. When a process p_i receives a message with timestamp C_{msg}, it executes the following actions:
  1. $C_i := max(C_i, C_{msg})$;
  2. execute **R1**;
  3. deliver the message.

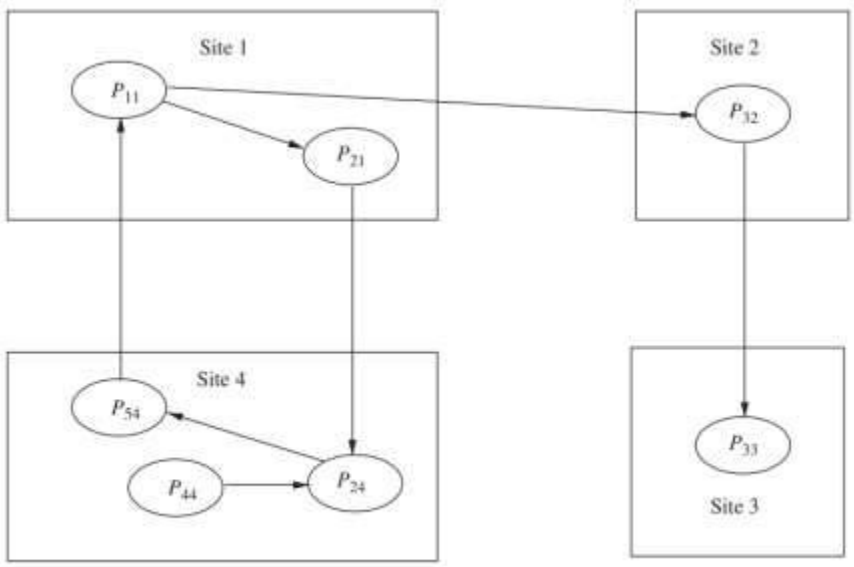
5) In the quorum-based approach, each site requests permission to execute the CS from a subset of sites (called a quorum). The quorums are formed in such a way that when two

sites concurrently request access to the CS, at least one site receives both the requests and this site is responsible to make sure that only one request executes the CS at any time.

6) **Wait-for graph (WFG)**

- In distributed systems, the state of the system can be modeled by directed graph, called a wait-for graph (WFG).
- In a WFG, nodes are processes and there is a directed edge from node P1 to node P2 if P1 is blocked and is waiting for P2 to release some resource.
- A system is deadlocked if and only if there exists a directed cycle or knot in the WFG

Figure 10.1 Example of a WFG.



7) Advantages:

- Shields programmer from Send/Receive primitives
- Single address space; simplififies passing-by-reference and passing complex data Structures.
- Exploit locality-of-reference when a block is moved
- DSM uses simpler software interfaces, and cheaper offff-the-shelf hardware. Hence cheaper than dedicated multiprocessor systems
- No memory access bottleneck, as no single bus
- Large virtual memory space
- DSM programs portable as they use common DSM programming interface

8)

• Let *e* be a non-deterministic event that occurs at process *p*

• *Depend*(*e*)

– the set of processes that are affected by a non-deterministic event *e*. This set consists of *p*, and any process whose state depends on the event *e*

according to Lamport' s *happened before* relation

• *Log*(*e*)

– the set of processes that have logged a copy of *e*'s determinant in their volatile memory

• *Stable*(*e*)

– a predicate that is true if *e*'s determinant is logged on the stable storage

• *always-no-orphans* condition

– $\forall(e) : \neg Stable(e) \Rightarrow Depend(e) \subseteq Log(e)$

9) Byzantine Agreement (single source has an initial value)

Agreement: All non-faulty processes must agree on the same value.

Validity: If the source process is non-faulty, then the agreed upon value by all the non-faulty processes must be the same as the initial value of the source.

Termination: Each non-faulty process must eventually decide on a value.

10)

PART-B

11) (a)

- There are several models of the service provided by communication networks, namely, FIFO, Non-FIFO, and causal ordering.
- In the FIFO model, each channel acts as a first-in first-out message queue and thus, message ordering is preserved by a channel.
- In the non-FIFO model, a channel acts like a set in which the sender process adds messages and the receiver process removes messages from it in a random order.

- The "causal ordering" model is based on Lamport's "happens before" relation.
- A system that supports the causal ordering model satisfies the following property:

  CO: For any two messages m_{ij} and m_{kj}, if $send(m_{ij}) \longrightarrow send(m_{kj})$, then $rec(m_{ij}) \longrightarrow rec(m_{kj})$.
- This property ensures that causally related messages destined to the same destination are delivered in an order that is consistent with their causality relation.
- Causally ordered delivery of messages implies FIFO message delivery. (Note that CO $\subset$ FIFO $\subset$ Non-FIFO.)
- Causal ordering model considerably simplifies the design of distributed algorithms because it provides a built-in synchronization.

(b)

<u>Chandy–Lamport algorithm</u>

- The Chandy-Lamport algorithm uses a control message, called a marker.
- After a site has recorded its snapshot, it sends a marker along all of its outgoing channels before sending out any more messages.
- Since channels are FIFO, a marker separates the messages in the channel into those to be included in the snapshot (i.e., channel state or process state) from those not to be recorded in the snapshot.

The algorithm

- A process initiates snapshot collection by executing the marker sending rule by which it records its local state and sends a marker on each outgoing channel
- A process executes the marker receiving rule on receiving a marker.
- If the process has not yet recorded its local state, it records the state of the channel on which the marker is received as empty and executes the marker sending rule to record its local state Otherwise, the state of the incoming channel on which the marker is received is recorded

- The algorithm can be initiated by any process by executing the marker sending rule.
- The algorithm terminates after each process has received a marker on all of its incoming channels.
- The recorded local snapshots can be put together to create the global snapshot

Marker sending rule for process p_i

(1) Process p_i records its state.
(2) For each outgoing channel C on which a marker has not been sent, p_i sends a marker along C before p_i sends further messages along C.

Marker receiving rule for process p_j
On receiving a marker along channel C:
  if p_j has not recorded its state **then**
    Record the state of C as the empty set
    Execute the "marker sending rule"
  else
    Record the state of C as the set of messages received along C after p_j's state was recorded and before p_j received the marker along C

Algorithm 4.1 The Chandy–Lamport algorithm.

OR

12) (a)

The following functions must be addressed when designing and building a distributed system:

1. Communication
2. Processes
3. Naming
4. Synchronization
5. Data storage and access
6. Consistency and replication
7. Fault tolerance
8. Security
9. Applications Programming Interface (API) and transparency
10. Scalability and modularity

(b)

Communication Systems: 1. Message Passing , 2. Shared Memory , Blocking/non-blocking, synchronous/asynchronous primitives

- Send() and Receive() primitives are used to send and receive messages

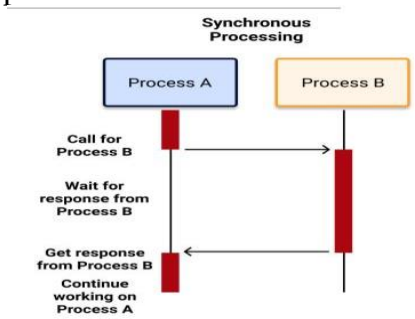
- A Send primitive has at least two parameters – the destination, and the buffer in the user space, containing the data to be sent
- Receive primitive has at least two parameters –source from which the data is to be received, and the user buffer into which the data is to be received.
- There are two ways of sending data when the Send primitive is invoked –
    1. the buffered option
    2. the unbuffered option.

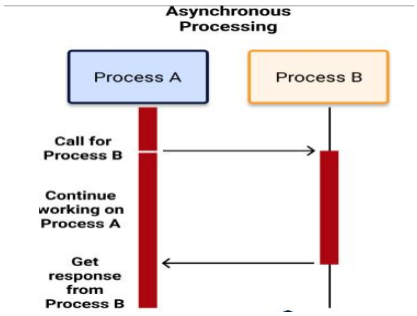
Synchronous primitives

A Send or a Receive primitive is synchronous if both the Send() and Receive() handshake with each other. The processing for the Send primitive completes only after the invoking processor learns that the other corresponding Receive primitive has also been invoked and that the receive operation has been completed

Asynchronous primitives

A Send primitive is said to be asynchronous if control returns back to the invoking process after the data item to be sent has been copied out of the user-specified buffer





13) (a)

<u>The bully algorithm</u>

- Process with highest id will be the coordinator
- There are three types of message in this algorithm:
- An election message is sent to announce an election;
- An answer message is sent in response to an election message
- A coordinator message is sent to announce the identity of the elected process.
- The process that knows it has the highest identifier can elect itself as the coordinator simply by sending a coordinator message to all processes with lower identifiers.
- On the other hand, a process with a lower identifier can begin an election by sending an election message to those processes that have a higher identifier and awaiting answer messages in response.
- If none arrives within time T, the process considers itself the coordinator and sends a coordinator message to all processes with lower identifiers announcing this.
- Otherwise, the process waits a further period T for a coordinator message

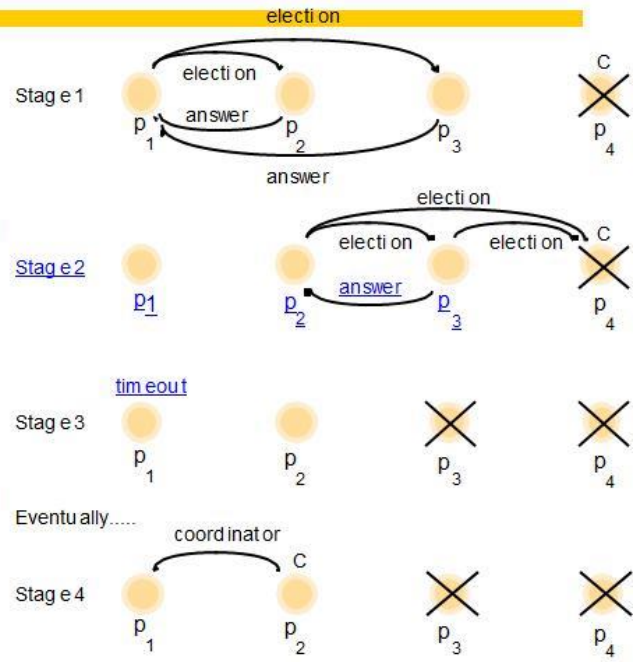
to arrive from the new coordinator.

- If a process p i receives a coordinator message, it sets its variable elected i to the identifier of the coordinator contained within it and treats that process as the coordinator.
- If a process receives an election message, it sends back an answer message and begins another election – unless it has begun one already.
- When a process, P, notices that the coordinator is no longer responding to requests, it initiates an election.

● P sends an ELECTION message to all processes with higher no.
● If no one responds, P wins the election and becomes a coordinator.
● If one of the higher-ups answers, it takes over.
● P' s job is done. When a process gets an ELECTION message from one of its lower-numbered colleagues:
● Receiver sends an OK message back to the sender to indicate that he is alive and will take over.
● Receiver holds an election, unless it is already holding one.
● Eventually, all processes give up but one, and that one is the new coordinator.
● The new coordinator announces its victory by sending all processes a message telling them that starting immediately it is the new coordinator.
If a process that was previously down comes back:
● It holds an election.
● If it happens to be the highest process currently running, it will win the election and take over the coordinator' s job.
● Biggest guy" always wins and hence the name "bully" algorithm.

The bully algorithm

1. The process begins an election by sending an election message to these processes that have a higher ID and awaits an answer in response.
2. If none arrives within time T, the process considers itself the coordinator and sends coordinator message to all processes with lower identifiers.
3. Otherwise, it waits a further time T' for coordinator message to arrive. If none, begins another election.
4. If a process receives a coordinator message, it sets its variable $elected_i$ to be the coordinator ID.
5. If a process receives an election message, it sends back an answer message and begins another election unless it has begun one already.



(b)

- The algorithm uses the fact that a consistent snapshot of a distributed system captures stable properties.
- Termination of a distributed computation is a stable property.
- Thus, if a consistent snapshot of a distributed computation is taken after the distributed computation has terminated, the snapshot will capture the termination of the computation.
- The algorithm assumes that there is a logical bidirectional communication channel between every pair of processes.
- Communication channels are reliable but non-FIFO. Message delay is arbitrary but finite.

Informal description

- The main idea behind the algorithm is as follows:
- when a computation terminates, there must exist a unique process which became idle last.
- When a process goes from active to idle, it issues a request to all other processes to take a local snapshot, and also requests itself to take a local snapshot.

- When a process receives the request, if it agrees that the requester became idle before itself, it grants the request by taking a local snapshot for the request.
- A request is said to be successful if all processes have taken a local snapshot for it.
- The requester or any external agent may collect all the local snapshots of a request.
- If a request is successful, a global snapshot of the request can thus be obtained and the recorded state will indicate termination of the computation, viz.,
- In the recorded snapshot, all the processes are idle and there is no message in transit to any of the processes

OR

14) (a)

Chandy–Lamport algorithm

- The Chandy-Lamport algorithm uses a control message, called a marker.
- After a site has recorded its snapshot, it sends a marker along all of its outgoing channels before sending out any more messages.
- Since channels are FIFO, a marker separates the messages in the channel into those to be included in the snapshot (i.e., channel state or process state) from those not to be recorded in the snapshot.

The algorithm

- A process initiates snapshot collection by executing the marker sending rule by which it records its local state and sends a marker on each outgoing channel
- A process executes the marker receiving rule on receiving a marker.
- If the process has not yet recorded its local state, it records the state of the channel on which the marker is received as empty and executes the marker sending rule to record its local state Otherwise, the state of the incoming channel on which the marker is received is recorded
- The algorithm can be initiated by any process by executing the marker sending rule.
- The algorithm terminates after each process has received a marker on all of its incoming channels.
- The recorded local snapshots can be put together to create the global snapshot.

Marker sending rule for process p_i

(1) Process p_i records its state.
(2) For each outgoing channel C on which a marker has not been sent, p_i sends a marker along C before p_i sends further messages along C.

Marker receiving rule for process p_j
On receiving a marker along channel C:
if p_j has not recorded its state **then**
Record the state of C as the empty set
Execute the "marker sending rule"
else
Record the state of C as the set of messages received along C after p_j's state was recorded and before p_j received the marker along C

Algorithm 4.1 The Chandy–Lamport algorithm.

(b)

- The algorithm assumes there are N processes Pi, $0 \leq i \leq N$, which are modelled as the nodes i, $0 \leq i \leq N$, of a fixed connected undirected graph.
- The edges of the graph represent the communication channels, through which a process sends messages to neighbouring processes in the graph.
- The algorithm uses a fixed spanning tree of the graph with process P0 at its root which is responsible for termination detection
- Process P0 communicates with other processes to determine their states and the messages used for this purpose are called signals.
- All leaf nodes report to their parents, if they have terminated.
- A parent node will similarly report to its parent when it has completed processing and all of its immediate children have

terminated, and so on.

- The root concludes that termination has occurred, if it has terminated and all of its immediate children have also terminated.
- The termination detection algorithm generates two waves of signals moving inward and outward through the spanning tree.
- Initially, a contracting wave of signals, called tokens, moves inward from leaves to the root.
- If this token wave reaches the root without discovering that termination has occurred, the root initiates a second outward wave of repeat signals.
- As this repeat wave reaches leaves, the token wave gradually forms and starts moving inward again.
- This sequence of events is repeated until the termination is detected

Definitions

1. Tokens: a contracting wave of signals that move inward from the leaves to the root.
2. Repeat signal: if a token wave fails to detect termination, node $P0$ initiates another round of termination detection by sending a signal called Repeat, to the leaves.
3. The nodes which have one or more tokens at any instant form a set S.
4. A node j is said to be outside of set S if j does not belong to S and the path (in the tree) from the root to j contains an element of S. Every path from the root to a leaf may not contain a node of S.
5. Note that all nodes outside S are idle. This is because, any node that terminates, transmits a token to its parent. When a node transmits the token, it goes out of the set S.

- Initially, each leaf process is given a token.
- Each leaf process, after it has terminated, sends its token to its

parent.

- When a parent process terminates and after it has received a token from each of its children, it sends a token to its parent.
- This way, each process indicates to its parent process that the subtree below it has become idle.
- In a similar manner, the tokens get propagated to the root.
- The root of the tree concludes that termination has occurred, after it has become idle and has received a token from each of its children.

The algorithm description

The algorithm works as follows:

1. Initially, each leaf process is provided with a token. The set S is used for book-keeping to know which processes have the token. Hence S will be the set of all leaves in the tree.
2. Initially, all processes and tokens are white. As explained above, coloring helps the root know if a message-passing was involved in one of the subtrees.
3. When a leaf node terminates, it sends the token it holds to its parent process.
4. A parent process will collect the token sent by each of its children. After it has received a token from all of its children and after it has terminated, the parent process sends a token to its parent.
5. A process turns black when it sends a message to some other process. This coloring scheme helps a process remember that it has sent a message. When a process terminates, if its is black, it sends a black token to its parent.

6. A black process turns back to white after it has sent a black token to its parent.
7. A parent process holding a black token (from one of its children), sends only a black token to its parent, to indicate that a message-passing was involved in its subtree.
8. Tokens are propagated to the root in this fashion. The root, upon receiving a black token, will know that a process in the tree had sent a message to some other process. Hence, it restarts the algorithm by sending a Repeat signal to all its children.
9. Each child of the root propagates the Repeat signal to each of its children and so on, until the signal reaches the leaves.
10. The leaf nodes restart the algorithm on receiving the Repeat signal.
11. The root concludes that termination has occurred, if:
    (a) it is white;
    (b) it is idle; and
    (c) it has received a white token from each of its children.

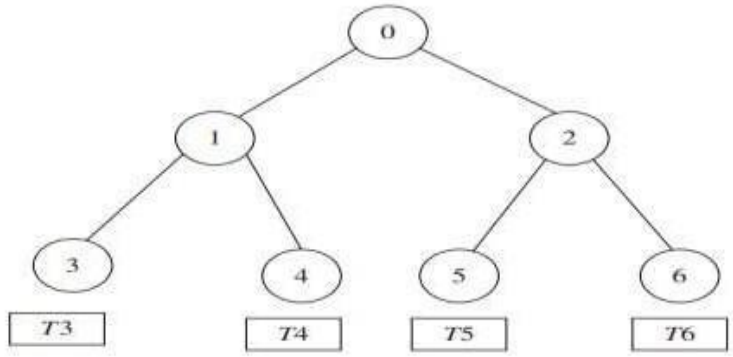


Figure 7.2 All leaf nodes have tokens. S = {3, 4, 5, 6}.

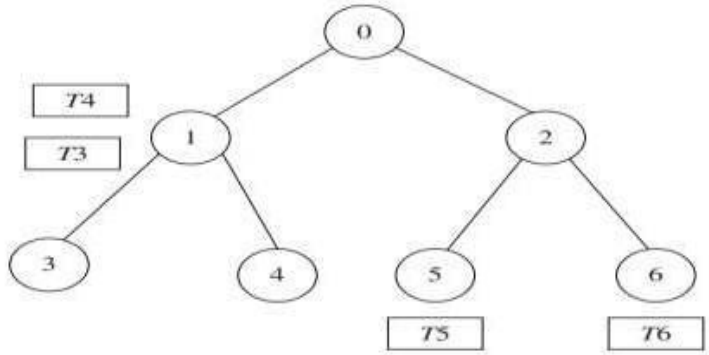


Figure 7.3 Nodes 3 and 4 become idle. $S = \{1, 5, 6\}$.

Figure 7.4 Node 1 becomes idle. $S = \{0, 5, 6\}$.

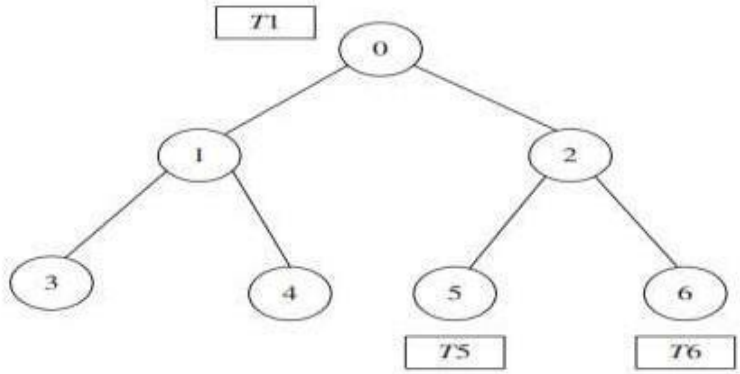


Figure 7.5 Node 5 sends a message to node 1.

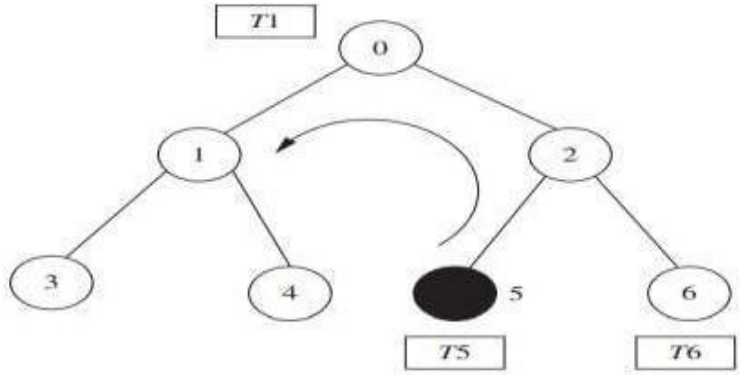


Figure 7.6 Nodes 5 and 6 become idle. $S = \{0, 2\}$.

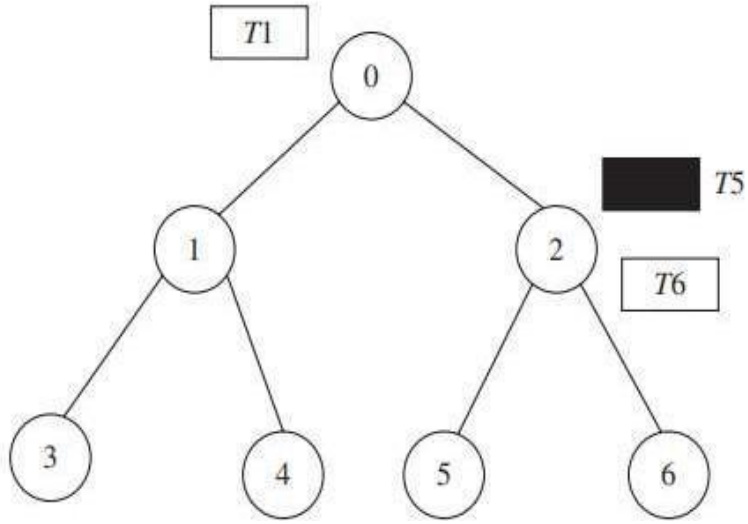
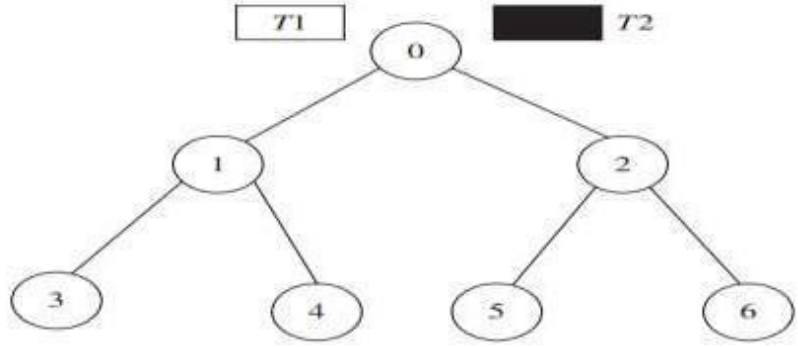


Figure 7.7 Node 2 becomes idle. $S = \{0\}$. Node 0 initiates a repeat signal.



We now present an example to illustrate the working of the algorithm.

1. Initially, all nodes 0 to 6 are white (Figure 7.2). Leaf nodes 3, 4, 5, and 6 are each given a token. Node 3 has token $T3$, node 4 has token T4, node 5 has token $T5$, and node 6 has token $T6$. Hence, S is {3, 4, 5, 6}.
2. When node 3 terminates, it transmits $T3$ to node 1. Now S changes to 1, 4, 5, 6. When node 4 terminates, it transmits $T4$ to node 1 (Figure 7.3). Hence, S changes to {1, 5, 6}.
3. Node 1 has received a token from each of its children and, when it terminates, it transmits a token $T1$ to its parent (Figure 7.4). S changes to {0, 5, 6}.
4. After this, suppose node 5 sends a message to node 1, causing node 1 to again become active (Figure 7.5). Since node 5 had already sent a token to its parent node 0 (thereby making node 0 assume that node 5 had terminated), the new message makes the system inconsistent as far as termination detection is concerned. To deal with this, the algorithm executes the following steps.
5. Node 5 is colored black, since it sent a message to node 1.

15) (a)

- Lamport developed a distributed mutual exclusion algorithm as an illustration of his clock synchronization scheme
- The algorithm is fair in the sense that a request for CS are executed in the order of their timestamps and time is determined by logical clocks.
- When a site processes a request for the CS, it updates its local clock and assigns the request a timestamp.
- The algorithm executes CS requests in the increasing order of timestamps.
- Every site Si keeps a queue, request_queuei, which contains mutual exclusion requests ordered by their timestamps.
- This algorithm requires communication channels to deliver messages in FIFO order.

Requesting the critical section

- When a site S_i wants to enter the CS, it broadcasts a REQUEST(ts_i, i) message to all other sites and places the request on $request_queue_i$. ((ts_i, i) denotes the timestamp of the request.)
- When a site S_j receives the REQUEST(ts_i, i) message from site S_i, it places site S_i's request on $request_queue_j$ and returns a timestamped REPLY message to S_i.

Executing the critical section

Site S_i enters the CS when the following two conditions hold:

L1: S_i has received a message with timestamp larger than (ts_i, i) from all other sites.

L2: S_i's request is at the top of $request_queue_i$.

Releasing the critical section

- Site S_i, upon exiting the CS, removes its request from the top of its request queue and broadcasts a timestamped RELEASE message to all other sites.
- When a site S_j receives a RELEASE message from site S_i, it removes S_i's request from its request queue.

Algorithm 9.1 Lamport's algorithm.

(b)

- Maekawa's algorithm was the first quorum-based mutual exclusion algorithm.
- This algorithm requires delivery of messages to be in the order they are sent between every pair of sites

Requesting the critical section:

(a) A site S_i requests access to the CS by sending REQUEST(i) messages to all sites in its request set R_i.

(b) When a site S_j receives the REQUEST(i) message, it sends a REPLY(j) message to S_i provided it hasn't sent a REPLY message to a site since its receipt of the last RELEASE message. Otherwise, it queues up the REQUEST(i) for later consideration.

Executing the critical section:

(c) Site S_i executes the CS only after it has received a REPLY message from every site in R_i.

Releasing the critical section:

(d) After the execution of the CS is over, site S_i sends a RELEASE(i) message to every site in R_i.

(e) When a site S_j receives a RELEASE(i) message from site S_i, it sends a REPLY message to the next site waiting in the queue and deletes that entry from the queue. If the queue is empty, then the site updates its state to reflect that it has not sent out any REPLY message since the receipt of the last RELEASE message.

Algorithm 9.5 Maekawa's algorithm.

OR

16) (a)

<u>The Ricart–Agrawala algorithm</u>

- The Ricart–Agrawala algorithm assumes that the communication channels are FIFO.
- The algorithm uses two types of messages: REQUEST and REPLY.
- A process sends a REQUEST message to all other processes to request their permission to enter the critical section.
- A process sends a REPLY message to a process to give its permission to that process.
- Processes use Lamport-style logical clocks to assign a timestamp to critical section requests.
- Timestamps are used to decide the priority of requests in case of conflict
- If a process pi that is waiting to execute the critical section receives a REQUEST message from process pj,
- then if the priority of pj's request is lower, pi defers the REPLY to pj and sends a REPLY message to pj only after executing the CS for its pending request.
- Otherwise, pi sends a REPLY message to pj immediately, provided it is currently not executing the CS.
- Each process pi maintains the request-deferred array, RDi, the size of which is the same as the number of processes in the system.
- Initially, ∀i ∀j: Rdi[j] = 0.

Requesting the critical section

(a) When a site S_i wants to enter the CS, it broadcasts a timestamped REQUEST message to all other sites.

(b) When site S_j receives a REQUEST message from site S_i, it sends a REPLY message to site S_i if site S_j is neither requesting nor executing the CS, or if the site S_j is requesting and S_i's request's timestamp is smaller than site S_j's own request's timestamp. Otherwise, the reply is deferred and S_j sets $RD_j[i] := 1$.

Executing the critical section

(c) Site S_i enters the CS after it has received a REPLY message from every site it sent a REQUEST message to.

Releasing the critical section

(d) When site S_i exits the CS, it sends all the deferred REPLY messages: $\forall j$ if $RD_i[j] = 1$, then sends a REPLY message to S_j and sets $RD_i[j] := 0$.

Algorithm 9.2 The Ricart–Agrawala algorithm.

(b)

Models of deadlocks

Distributed systems allow many kinds of resource requests.

A process might require a single resource or a combination of resources for its execution

Models of deadlocks introduces a hierarchy of request models starting with very restricted forms to the ones with no restrictions

1. The single-resource model

- The single-resource model is the simplest resource model in a distributed system, where a process can have at most one outstanding request for only one unit of a resource.
- Since the maximum out-degree of a node in a WFG for the single resource model can be 1, the presence of a cycle in the WFG shall indicate that there is a deadlock

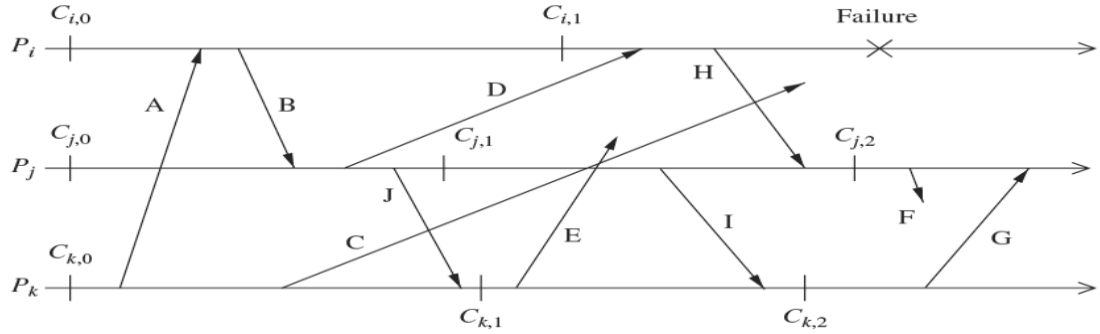
2. The AND model

- In the AND model, a process can request more than one resource simultaneously and the request is satisfied only after all the requested resources are granted to the process.
- The requested resources may exist at different locations.
- The out degree of a node in the WFG for AND model can be more than 1.
- The presence of a cycle in the WFG indicates a deadlock in the AND model.

3. The OR model

- In the OR model, a process can make a request for numerous resources simultaneously and the request is satisfied if any one of the requested resources is granted.
- The requested resources may exist at different locations.
- If all requests in the WFG are OR requests, then the nodes are called OR nodes.
- Presence of a cycle in the WFG of an OR model does not imply a deadlock in the OR model.

17) (a)



• Checkpoints : $\{Ci,0, Ci,1\}$, $\{Cj,0, Cj,1, Cj,2\}$, and $\{Ck,0, Ck,1, Ck,2\}$
• Messages : A - J
• The restored global consistent state : $\{Ci,1, Cj,1, Ck,1\}$
• The rollback of process Pi to checkpoint $Ci,1$ created an orphan message H
• Orphan message I is created due to the roll back of process Pj to checkpoint $Cj,1$
• Messages C, D, E, and F are potentially problematic
  – Message C: a delayed message
  – Message D: a lost message since the send event for D is recorded in the restored state for Pj , but the receive event has been undone at process Pi.
  – Lost messages can be handled by having processes keep a message log of all the sent messages
  – Messages E, F: delayed orphan messages. After resuming execution from their checkpoints, processes will generate both of these messages

(b)

Mutual Exclusion:- we know that when no process is executing in its critical section, a process with the lowest number is allowed to enter its critical section. Suppose two processes have the same token number. In that case, the process with the lower process ID among these is selected as the process ID of each process is distinct, so at a particular time, there will be only one process executing in its critical section. Thus the requirement of mutual Exclusion is met.

Progress: After selecting a token, a waiting process checks whether any other waiting process has higher priority to enter its critical section. If there is no such process, P will immediately enter its critical section. Thus meeting progress requirements.

Bounded Waiting: As awaiting, the process will enter its critical section when no other process is in its critical section and

- If its token number is the smallest among other waiting processes.
- If token numbers are the same, it has the lowest process ID among other waiting processes

OR

18) (a)

In asynchronous distributed systems, a global state obtained through remote

observations could be obsolete, incomplete, or inconsistent. Informally, a global state is inconsistent **if it could never have been constructed by an idealized observer that is external to the system**.

- **T**

(b) Log-based Rollback Recovery

• A log-based rollback recovery makes use of deterministic and nondeterministic events in a computation.

• Deterministic and Non-deterministic events

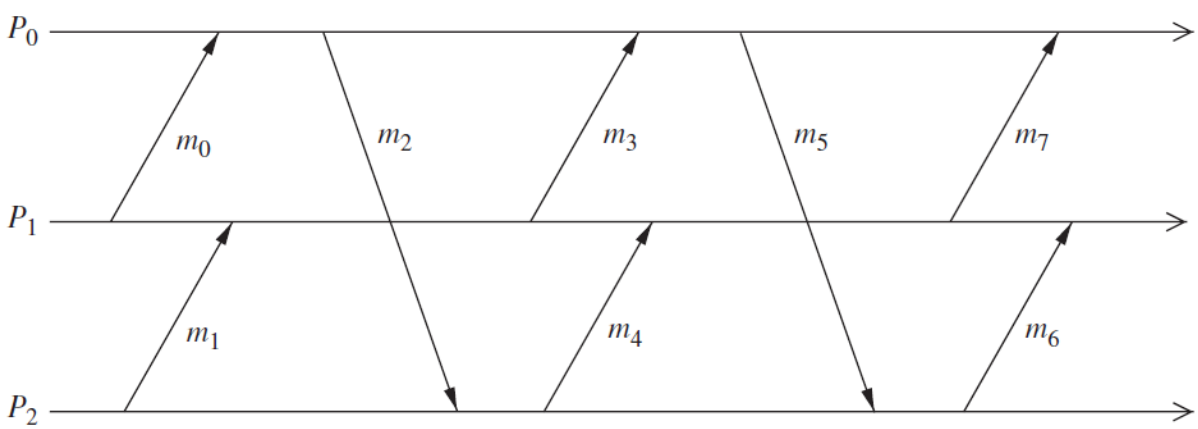
– Non-deterministic events can be the receipt of a message from another process or an event internal to the process

– a message send event is *not* a non-deterministic event.

– the execution of process *P*0 is a sequence of four deterministic intervals

– Log-based rollback recovery assumes that all non-deterministic events can be identified and their corresponding determinants can be logged into the stable storage

– During failure-free operation, each process logs the determinants of all non-deterministic events that it observes onto the stable storage



19) (a)

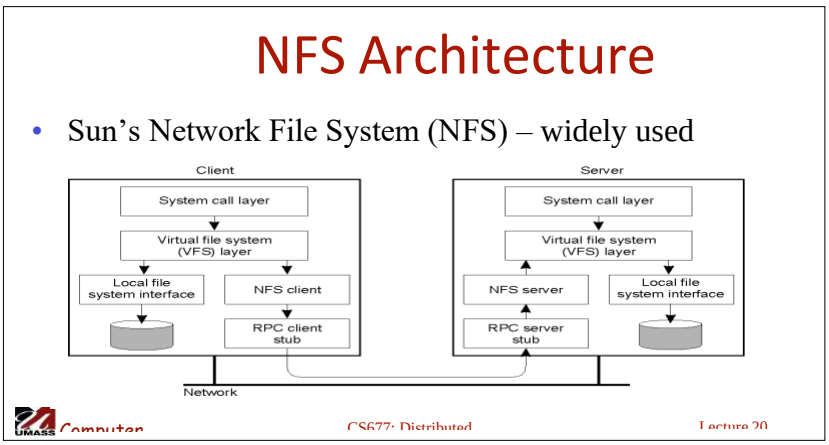
- Up to f (< n) crash failures possible.
- In f + 1 rounds, at least one round has no failures.
- Now justify: agreement, validity, termination conditions are satisfified.
- Complexity: $O(f + 1)n2$ messages
- f + 1 is lower bound on number of rounds

(global constants)
integer: f ; // maximum number of crash failures tolerated

(local variables)
integer: $x \leftarrow -$ local value;
(1) Process Pi ($1 \leq i \leq n$) executes the Consensus algorithm for up to f crash failures:
(1a) **for** *round* **from** 1 **to** $f + 1$ **do**
(1b) **if** the current value of x has not been broadcast **then**
(1c) **broadcast**(x);
(1d) $yj \leftarrow -$ value (if any) received from process j in this round;
(1e) $x \leftarrow - min(x, yj)$;
(1f)**output** x as the consensus value.

(b) **There are various features to a DFS, such as the following:**

- Transparency. Transparency is a security mechanism that shields details of one file system from other file systems and users. ...
- Performance. ...
- Scalability. ...
- High availability. ...
- Data integrity. ...
- High reliability. ...
- Security. ...
- User mobility.

20 a) in AFS, the server keeps track of which files are opened by which clients (as was not in the case of NFS). In other words, **AFS has stateful servers, whereas NFS has stateless servers**.

b)